

ROBLOX BUSINESS · JUSTIN & HUNTER · V1.1

The Game Factory

Business plan and technical architecture for a system that builds many Roblox games — and gets better at it every time.

A game is disposable. The factory is not. Every title that ships or dies must hand something back — a rule that hardened, a bar that rose, a piece of engine, a finding with its context attached — so the fourth game starts smarter than the third. This document says what that factory is, how its parts connect, where it could hurt a good game, and what is still missing.

WRITTEN	12 September 2026 · v1.1 same day, after the second pass (economy simulated, contradictions fixed, digest rewired)
FOR	Justin (owner) · Hunter (design, playtest operations) · every future Claude and Grok session
GOVERNED BY	docs/ROBLOX_SUCCESS_LOGIC.md — if this document fights that file, that file wins
CANONICAL	roblox-pipeline/docs/FACTORY_PLAN.md · this PDF is a rendering
COMPILED FROM	the doctrine, PIPELINE, SYSTEM-MAP, GAME_FACTORY, SYSTEM_ARCHITECTURE, run log 01, the idea log, the evidence ledger, and today's session
RE-CHECK BY	12 November 2026 — platform facts go stale in months

PART 0

Why this document exists

You asked for it in one breath and it was the right ask: *"I need to understand that what we have created will improve games and learn from my work and not detract, limit, or distract from a good game project or opportunity."*

Here is why you want it, said back to you plainly. You are not a programmer. Over one long day you approved a great deal of infrastructure you cannot read directly — Python hooks that refuse code, a ledger schema, an architecture document, an autonomy contract, a changelog. Each piece was argued for on its own. Nobody has yet shown you the whole thing standing up together and asked the only question that matters: **does this make games better, or does it make a machine that admires itself?**

Your fear all day has been consistent and correct: that the system becomes the thing that gets in the way. That rules make games boring. That AI confuses a process. That Claude gets stuck on an old process. That goalposts hold you back. This document takes that fear as its test. Every mechanism in Part 2 is judged in Part 4 by two questions — *how could this hurt a good game?* and *what stops it?* — and where the honest answer is "nothing stops it yet," it says so.

How to read it

- **Part 1** is the business plan — what we are building, for whom, how it makes money, and what the market actually rewards. No code.
- **Part 2** is the system — every layer, loop, gate and rule, each with a diagram. Skim the figures first; the text explains them.
- **Part 3** is the roster — every agent, mechanism and task, and how they connect. You asked for this list specifically.
- **Part 4** is the re-examination — where the system could detract, the contradictions found between our own documents, and the risks that outrank everything built today.
- **Part 5** is status and the build order — what exists, what is missing, what happens next, and the few decisions that are genuinely yours.
- **The glossary** at the back defines every technical word used here in one line. Nothing in this document requires reading code.

The one-paragraph answer

Yes, the system will improve games and learn from your work — *if* it is now used to build one. Everything built today is scaffolding around a build loop that does not yet exist and a game that has not yet met a stranger. The scaffolding is sound, and Part 4 shows where it could bite. But the single largest risk to this project as of tonight is not any rule in it. It is that the system becomes the project. The test of the factory is a game, and the next session should build one through it.

PART 1 · BUSINESS PLAN

The intent

1.1 What we are building

A system that turns a four-paragraph description into a working Roblox game, asks the same questions and applies the same filters every time, learns from each title, and does it for many games across many projects. The product is the factory. Any one game is a test case.

This distinction is not cosmetic. It decides where effort goes. A studio that treats each game as the product rebuilds its saving system, its shop, its security and its process every time, differently, and learns nothing transferable. A factory builds those once, keeps them, and spends each new title's effort on the only things that are actually new: the idea, the hook, the theme, the tuning, the art.

1.2 Who is building it, and how the work divides

Three parties, with a division of labour that was clarified today and now governs everything:

WHO	BRINGS	DOES NOT BRING, AND IS NOT ASKED TO
Justin	Product ownership. The five gate decisions. The cut — removing what confuses. Every decision that spends money or ships to players. About twelve minutes a weekday.	Code. Generative taste — inventing a mechanic from nothing.
Hunter	Design input. Operating the comprehension test — recruiting kids who have never seen the game and running the script. Playtesting titles he did not help design.	Being the test subject for a game he helped make — he already knows the answers.
Claude	The only writer of code and the only hand in Studio. Research, design, economy, Luau, documentation, orchestration. Proposing every applicable idea. Filing evidence. Operating within the autonomy contract.	Publishing. Creating paid products. Killing or switching a title. Spending money. Any of those is a human click.
Grok	A coworker, not a competitor: research, art sheets, clip hooks, a second pass on design and economy, critique. Live X/Twitter discourse if Justin already has access.	Writing to the repo or the live Studio DataModel while Claude is. No new paid tier is assumed.

The operating model, in one line

The AI proposes everything applicable. Justin removes what is confusing. Neither role is the other one's job.

Justin said today that he is "not sure he can supply the taste." He is supplying it — the subtractive half, which is the scarcer one. Almost anyone can add a feature. Almost nobody cuts one. What he is not asked for is invention, and the system is built so he never has to be.

1.3 The market we are actually in

These facts come from the doctrine and pipeline documents, verified against Roblox's own material in September 2026. They will need re-verifying by November. They are the reason the system is shaped the way it is.

- **It is a power-law market.** Most published experiences earn nothing. Of creators who reach the payout programme, the median is on the order of \$1,500 a year. A thin slice earns real money; a thinner slice earns \$10k+ a month. AI makes production cheaper for everyone at once, so extra content buys no attention. Attention is the scarce thing.
- **Discovery now pays for return, not for clicks.** Roblox's 2026 recommendation rework weights day-1, week-1 and week-4 return, session quality and spend-days — and explicitly devalues thumbnail-driven first clicks. A 200-player community that comes back beats a 5,000-player bounce house.
- **The platform removes manipulative games, including #1.** *Steal an Egg* was the top game on the platform at roughly 800,000 concurrent players when it was taken down in August 2026 over a forced-engagement mechanic. Simulated gambling is banned at every rating. Paid random items need exact odds shown before purchase and are restricted outright in five countries.
- **The revenue proof points are generous games.** *Grow a Garden* (a 22-million-concurrent record) runs on offline growth that respects bedtime and Saturday events kids look forward to. *Steal a Brainrot* runs on drama kids clip for TikTok plus scheduled giveaways. The biggest games retain by being loved and monetise by being loved.
- **The audience is children, which is the most regulated corner of gaming.** The FTC took \$520 million from Epic over dark-pattern purchases and children's privacy. Parents control both the wallet and Roblox's parental spend controls. Every feature clears two bars: would Roblox allow it, and would we let Hunter pay for it without us in the room.
- **The money is small per unit and real in aggregate.** DevEx pays about \$0.0038 per Robux (\$0.0054 for spend from verified adults). 100,000 Robux is roughly \$380. Creator Rewards now pays per active spender who plays ten-plus minutes, and a share of the first \$100 spent by new players you bring to the platform — so retention and off-platform promotion are line items, not virtues.

- **Clone waves rise in weeks and fade in months.** *Steal a Fish*: launch, 192,000 peak concurrent, hundreds today — inside a few months. Enter weeks 0–3 or not at all. Speed is the moat, which is what the factory is for.

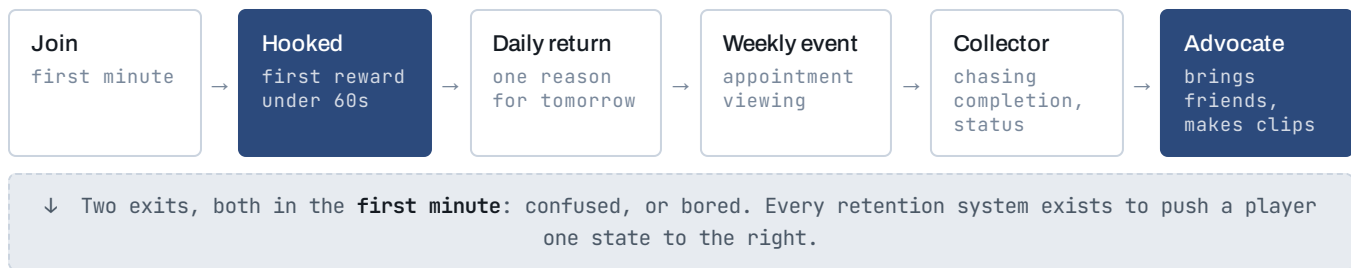


Figure 1 – The player lifecycle. Compiled from SYSTEM-MAP §6. The first sixty seconds decide almost everything, which is why the factory's quality bar is comprehension rather than feature count.

1.4 How a game in this factory makes money

Revenue is a lagging indicator of two things: retention, and a fair shop. The doctrine's ordering is locked and this plan does not soften it: **build a loop people reopen → measure it → patch it → clip it → spend last.**

Justin's own thesis on why kids pay is drawn below as a loop, because it is the operating principle of everything after launch. Kids do not buy a bare catalogue row. A scheduled event creates a specific want; the product attaches to that moment; the payoff is visible to other players or felt in play; and that is what earns the next purchase. Every product is classified against one of four drivers — accelerate fun they already have, show status others see, complete a collection, participate socially — and if it maps to none, it does not ship.

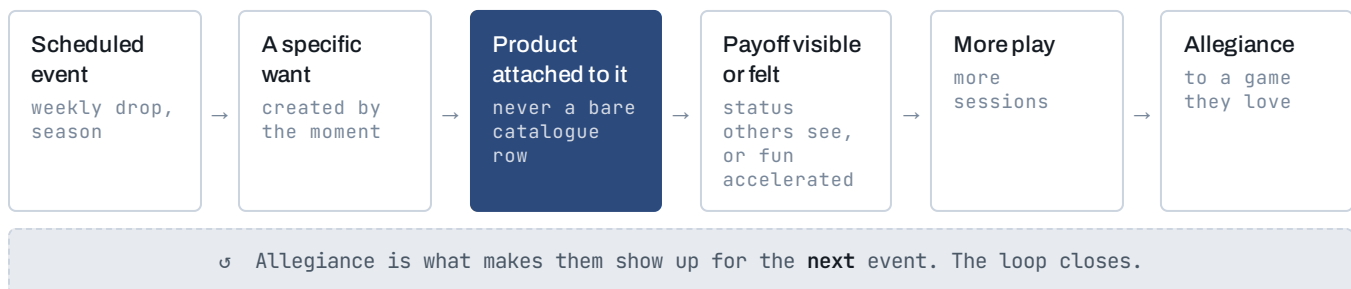


Figure 2 – The monetisation "why" loop. Justin's thesis from SYSTEM-MAP §4. It is also why the factory refuses the coercion catalogue — panic timers, pay-to-stop-suffering, guilt loops — not on ethics alone but because on this platform in 2026 generosity is the winning strategy.

1.5 Every game is three loops stacked

A pitch that cannot fill all three boxes will not retain, whatever its art looks like. The core loop is the second-to-second fun. The meta loop is the reason to come back tomorrow. The social loop is the retention engine — it is what turned *Steal a Brainrot* into 25 million

concurrent players, and Roblox's discovery system tracks intentional co-play directly.

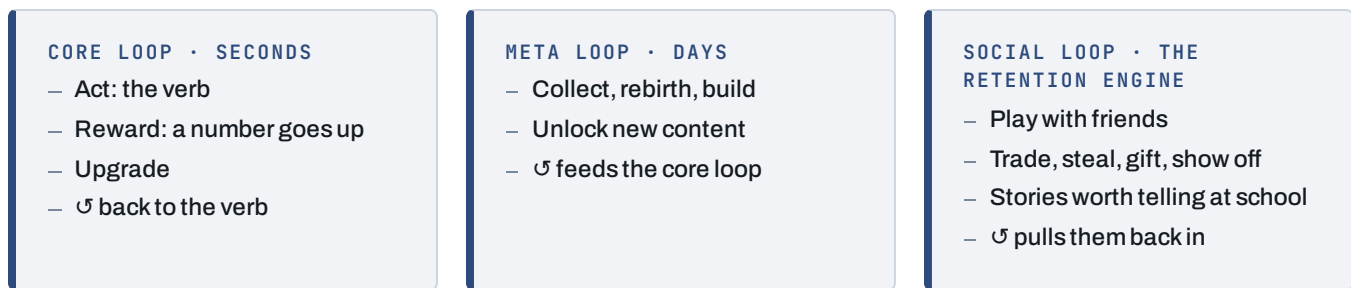


Figure 3 – The three nested loops. From SYSTEM-MAP §2. The complexity budget in Part 2 allows exactly one core verb and one return hook in a first slice; the social loop is the one that is hardest to fake and the one that matters most.

1.6 A portfolio, built one title at a time

Most titles will be killed at a gate. That is the design, not a failure. The factory only compounds when a title reaches a closeout, so killing is done *at a gate, on evidence, with a learnings record* — never mid-build because something newer looks fun. That second pattern, twelve half-games that never get a real first session, is bad process labelled as bad luck, and the doctrine names it as the failure to avoid.

"Many games" and "one active title" do not conflict. The factory supports any number of titles — live, parked, waiting — across any number of projects. The scarce resource is not the machine; it is build attention. One title in active build; everything else in some other state.

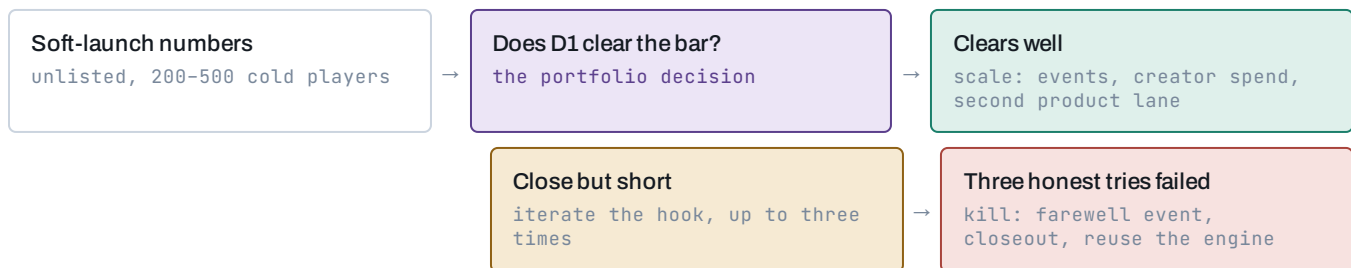


Figure 4 – Kill or scale. From SYSTEM-MAP §9, with one correction made in Part 4: the three documents that set D1 thresholds disagree with each other, and the doctrine's numbers govern. A fast funeral is a win; it frees the slot for the next wave.

1.7 Targets, used as a compass and not a prophecy

1. **Near-term:** a stranger completes the full earn → spend → progress loop with nobody in voice chat.
2. **Next:** the Gate B table mostly green — cold players return.
3. **Then:** DevEx-eligible earned Robux. Hobby money is a real win.

4. **Not a plan:** \$10k a month from the first publish. That tier exists and it is as rare as it was before AI; Claude and Grok only cut the hours in the script editor.

1.8 What it costs

Net additional spend required to operate the factory today: **\$0**. Claude and its scheduled tasks run inside the plan already paid for; Studio, its built-in MCP server, Cube 3D, the Creator Store's licensed music, and Roblox's native A/B Experiments are free and first-party. Grok Build and Grok Bots were evaluated and not adopted: at this scale they add subscription cost and coordination overhead without a capability Claude lacks, and would be revisited only at three-plus live games. Optional spend arrives with real gates: an image tool at $\leq \$10$ a month when a thumbnail matters at G4; a \$20–200 micro-creator test after Gate B; small sponsored tiles at Gate C, only after D1 has cleared the bar, and only on the icon that already won.

INTERLUDE

The same system, told as the life of one title

Part 2 explains the factory by component. This page explains it by time — what actually happens to one game from the four paragraphs to the closeout. If Part 2 is the wiring diagram, this is the story. Both describe the same machine.

Hour 0 — four paragraphs.

Justin describes the game in prose. The intake step drafts the six-question brief from it; he corrects, never fills. The ledger is asked what it already knows about this kind of game.



Hour 0–1 — expand, then cut.

Claude and Grok propose every applicable mechanic. Grok runs a confusion pass — "which of these would lose a ten-year-old?" The budget auto-cuts to the caps. Justin moves the line in two minutes. Everything below it goes to the parking list.



Hour 1–3 — the slice.

Content pack on the engine, hooks enforcing R1–R7 on every write, one core verb, one return hook, the onboarding artifact. Economy simulated before anyone plays. Two hypotheses written into the brief.



Day 1–3 — Gate A.

Hunter runs three fresh kids through the verbatim script. Justin does the ten-minute fun-check. Stop-and-patch until they can say what the game is and want a second go. The budget's two questions get answered. First player evidence enters the ledger.



Week 1 — art, then cold players.

Icon and thumbnail readable at 200px (G4). Then a listed publish with zero spend — Gate B — because an unlisted place meets no strangers. Two hundred to five hundred of them, and the retention table fills in.



Week 2–4 — iterate or kill.

D1 under the floor: rewrite the hook, up to three times. Still under: kill it at the gate, farewell event, closeout. D1 clears: G5, the human click, then shorts, micro-creators, and only then small ads on the icon that already won.



Month 2 onward — live-ops.

Monday digest, Friday numbers, one-line yes, one drop a week. Events give purchases their why. Monthly economy audit. The game is never finished; it is operated.



Closeout — the title pays back.

Hypotheses marked confirmed or refuted. Deviations recorded. Anything written twice moves into the engine. A bar that was beaten becomes the new floor. The next title starts here, not at zero.

Figure 16 — The life of a title. Every purple box is where a human stops the machine. Every other box runs on stated defaults. The last box is the only reason the factory exists.

PART 2 · TECHNICAL ARCHITECTURE

The system

Eight components. Each has a figure, a plain-language explanation, and a status. Nothing here requires reading code; the code that implements it is listed by path so a future session can find it.

2.1 The factory and the title are different things

The single most important idea in the architecture. A title is disposable by design. What must never be disposable is what the title leaves behind.

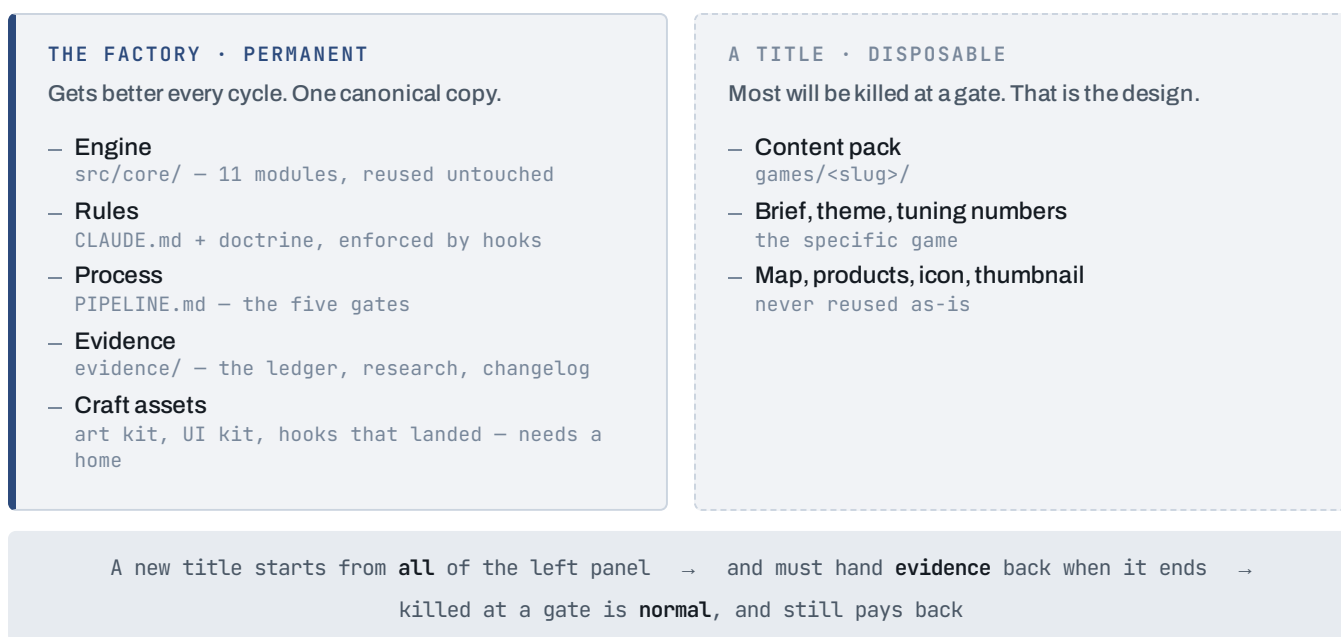


Figure 5 – Factory versus title. Rule: a title is finished when it has handed something back, not when it ships or dies. A title that taught the factory nothing was a wasted cycle even if it made money. Status: the boundary exists in code (engine/pack split, run 01) and is now enforced by hook R7.

2.2 Four layers of control

Everything that keeps quality up is one of four kinds of thing. They are not interchangeable, and the common mistake — the one that makes a system heavy without making games better — is trying to solve a higher layer with a lower one.

04

The cut

HUMAN

Not "invent the fun" — **remove what confuses**. What is left has to be something we would let Hunter play and pay for with nobody in the room.

Owner: Justin, from a ranked list · never automated

03

The learning loop

MISSING

Does game four start smarter than game three? The ledger that makes this possible exists as of today; the mechanisms that *feed* it — closeouts, the session-close hook, ledger-aware research — do not yet.

Owner: evidence/ — substrate built, feeders missing

02

Quality bars

PARTIAL

Numbers a build has to hit, so "is it good" is evidence rather than opinion. Gate B had hard bars already; Gate A — the gate that fires on every build — got its bars today via the comprehension test.

Owner: Gates A / B / C in the doctrine

01

Safety rails

BUILT

Binary, machine-enforced, no judgment. Can't be exploited, can't lose saves, can't get banned. **A game that passes every one of these can still be boring.**

Owner: seven hooks + a git backstop · automatic, every session, every title

Figure 6 — The four layers. Higher layers need judgment; lower layers need none. Layers 1-3 exist to make Layer 4 cheap and fast: get a playable thing in front of real people quickly, then tell the truth about it.

Is the evidence ledger a fifth "wisdom layer"? No — and the reason matters

A control layer has a veto: safety refuses a write, quality passes or fails a build, the cut removes. **The ledger has no veto and must never be given one.** The moment past results can block a new idea, the system becomes conservatism with a database — one finding from one game vetoing the second game — which is exactly the "stuck on an old process" failure Justin named. The ledger is the substrate Layer 3 runs on. Wisdom is not stored in it; wisdom is *promoted* out of it when a finding is reproduced by a second title and becomes a default.

2.3 The pipeline and its gates

One line in at the top — "*create a ___ game*" — and a live, operated game at the bottom. Five human decisions stop the machine; everything else runs on stated defaults, and silence means proceed for everything except publishing and paid products, which wait forever.

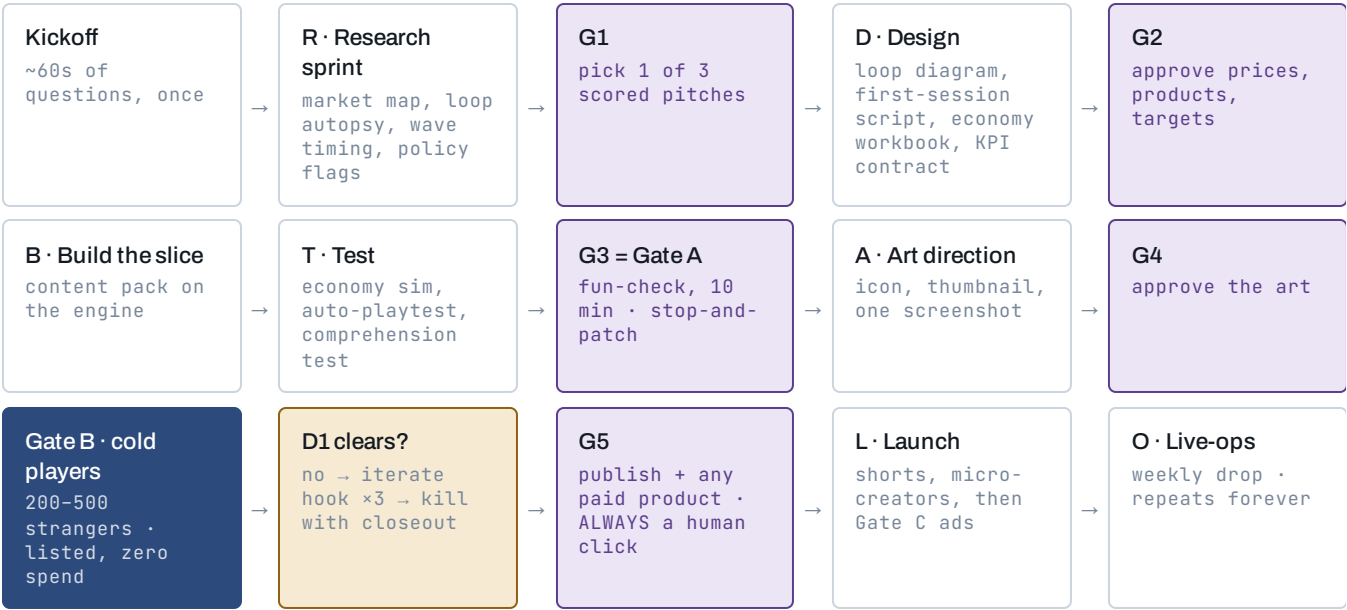


Figure 7 — The master pipeline. Compiled from SYSTEM-MAP §1 and PIPELINE §0. Purple boxes are the only places the machine stops for a human. Gate B has no G-number because the original plan assumed publish-then-promote; the doctrine inserted it: *cold-player proof happens before any money*.

DOCTRINE	PIPELINE	WHAT IT IS	WHAT PASSES IT
—	G1	Pick the pitch	Justin chooses one of three one-page pitches scored on session potential, spend clarity, build cost, wave timing, policy risk, differentiation, Hunter-testability
—	G2	Approve the economy	Prices, products, KPI targets — one spreadsheet, one reply
Gate A	G3 (+G4 art)	Internal playtest, no spend	The comprehension test (§2.4) plus: earn → spend → progress unassisted, one full session with no softlock or data loss
Gate B	(between G4 and G5)	Cold players — a listed publish with zero spend. An unlisted place meets no strangers; Roblox's own new-experience impressions are the cheapest cold traffic there is and exactly the algorithm we want to test. Listing is a G5 click	Median first session completes the loop + one spend; D1 return ~12%+ (the doctrine floor — PIPELINE's 20/25% are tiers above it); favourites up; a few organic buys with no pleading; stable
Gate C	after G5	Paid traffic	Small sponsored test on the winning icon only, capped. High cost-per-play with no return → kill the campaign; the ad did not fail, the game did
—	G5	Publish · create paid products	A human click. Never automated. Silence is not approval.

2.4 The build loop — describe, expand, prune

This is the stated purpose of the whole project, in Justin's words: *write four paragraphs describing a game, and have Claude and Grok put together a working model — asking the same questions and applying the same filters every time, so it moves fast.*

The risk that makes this hard is not that the rules are too strict. It is the opposite. **AI's failure mode on a creative brief is bloat.** Asked for a game, it adds every applicable idea, and the result is a slice nobody can read. A game rarely dies of being too plain; it dies of the player not understanding what to do in the first fifteen seconds. So the loop has an expansion step *and* a forced contraction step, and neither is optional.

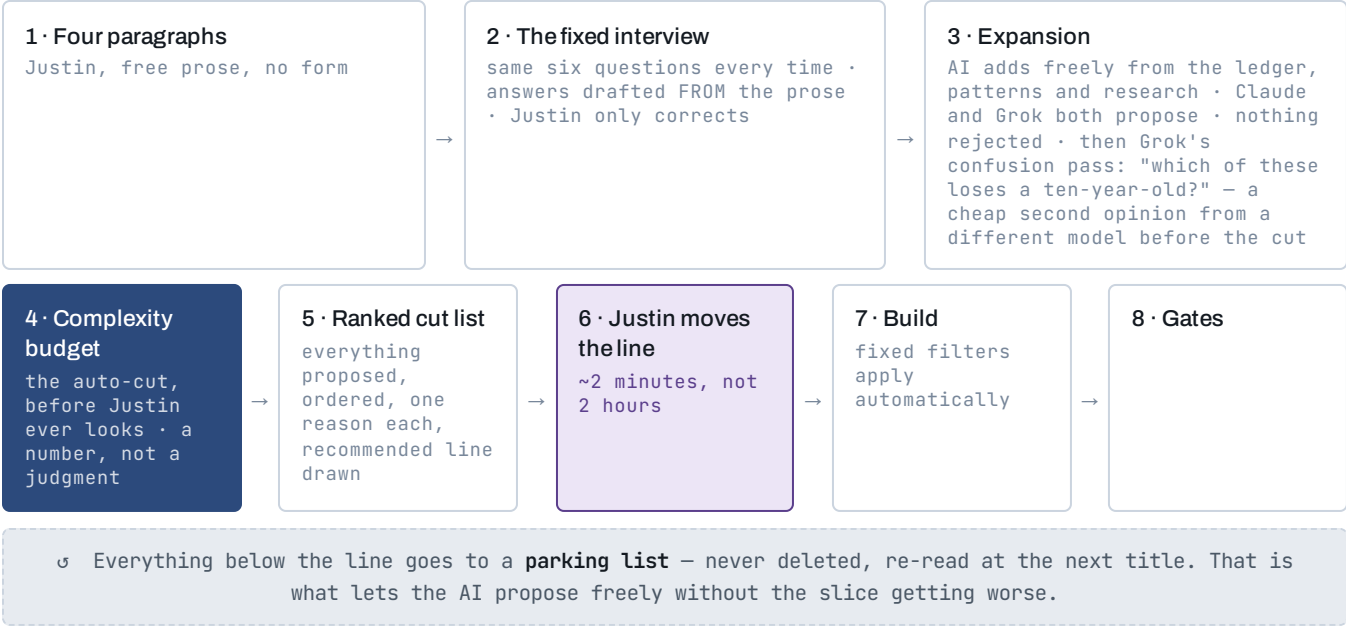


Figure 8 — The build loop. Half exists: the six-question brief is GAME_FACTORY §2 and has been used once. Missing: the paragraphs-to-brief step, the expansion inventory, the budget as code, the ranked cut list. This is gap 0 in Part 5.

THE COMPLEXITY BUDGET

A cap is a taste judgment wearing a number's clothing, so nobody is asked to be right about it today. The caps are **provisional defaults with an expiry**: they hold until the first Gate A, which asks two questions about the budget itself — did a cap block something the slice needed (loosen), and did testers get confused inside the caps (tighten). Both file evidence. A cap that survives two titles becomes a real default.

SLOT	FIRST SLICE ALLOWS	WHY THIS NUMBER
Core verb	exactly 1	The one thing the player does second to second
Return hook	exactly 1	Doctrine §4.5 — one, implemented well
Social / clip hook	at most 1	The thing a twelve-year-old films
Player-facing systems	at most 4	Shop, upgrade, zone, collection — count them

SLOT	FIRST SLICE ALLOWS	WHY THIS NUMBER
On-screen numbers	at most 3	A fourth counter is where the HUD stops being readable
Onboarding artifact	1, required	Not "no explanation" — see the comprehension bar

The budget constrains the *first slice*, not the game. Games grow through live-ops; the budget is about legibility at first contact, which is where games die. → [docs/BUDGET.md](#)

THE ONBOARDING AGENT AND THE COMPREHENSION BAR

Every game ships a walkthrough, intro or how-to, built by a dedicated agent — a standing requirement of the factory and a missing engine component today. The bar is not a stopwatch:

A ten-year-old who has never seen the game understands it quickly enough to want to keep playing.

"Fifteen seconds" sounds rigorous and measures the wrong thing — a game can be understood in five seconds and still be ignored. The real test is comprehension fast enough to produce engagement, and the only instrument for it is an actual ten-year-old. This gives Gate A the numbers it lacked:

GATE A CHECK	PASS CONDITION	HOW IT IS RUN
Goal comprehension	2 of 3 testers say in their own words what they are trying to do	Ask after first exposure. Do not explain first
First action	2 of 3 act meaningfully within 60s, unprompted	Watch. Say nothing
Pull	2 of 3 want a second go, unasked	Watch. Do not prompt. Do not ask if they liked it
Loop completion	Earn → spend → visible progress, unassisted	Existing doctrine bar
Stability	One full session, no softlock, no data loss	Existing doctrine bar

Subjects: three or more kids who have never seen the game and did not help design it. Operator: Hunter. Nobody: Justin. Verbatim answers matter more than the pass/fail — "*you collect the coins before the flood*" and "*you run from the water*" are both passes and describe two different games. → [skills/playtest-comprehension.md](#)

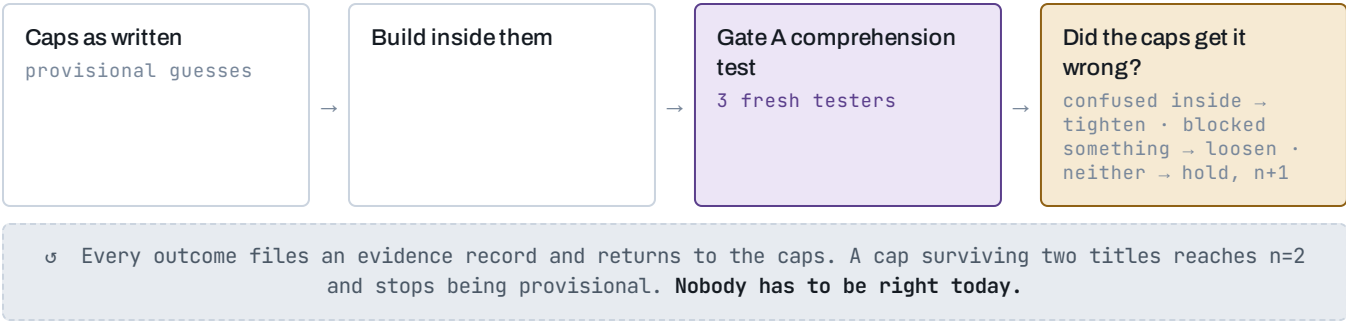


Figure 9 – The budget expiry loop. How the goalposts set themselves from evidence instead of from anyone's opinion. Claude may move a cap on its own when a Gate A result justifies it (autonomy contract, §2.7), logging the change.

2.5 The enforcement layer

The non-negotiables in `CLAUDE.md` stopped being sentences a long session can drift past and became code that refuses the write. These seven are the rules that are both *catastrophic* and *detectable in a diff* – they fail silently, in production, after a human already approved the build. Nothing here has an opinion about whether a game is fun.

ID	RULE	EFFECT	THE FAILURE IT PREVENTS
R1	Client-authoritative economy	BLOCK	One player edits their own balance; the economy dies overnight
R2	Raw remote handler	BLOCK	An unvalidated, unthrottled entry point. Everything binds through RemoteGuard
R3	ProcessReceipt without a Purchased ledger	BLOCK	Roblox retries receipts. Double-grant (we eat it) or drop it (a kid paid and got nothing)
R4	DataStore access outside DataService	BLOCK	A second writer without the session lock — this is how saves get wiped
R5	Gambling-shaped monetisation	WARN	Banned at every rating. A takedown risk, not a bug — needs a human decision
R6	Third-party IP	WARN	Moderation removes it, and the takedown lands <i>after</i> the title is worth something
R7	Engine module names a specific game	BLOCK	The engine stops being reusable and nothing is portable to a second project

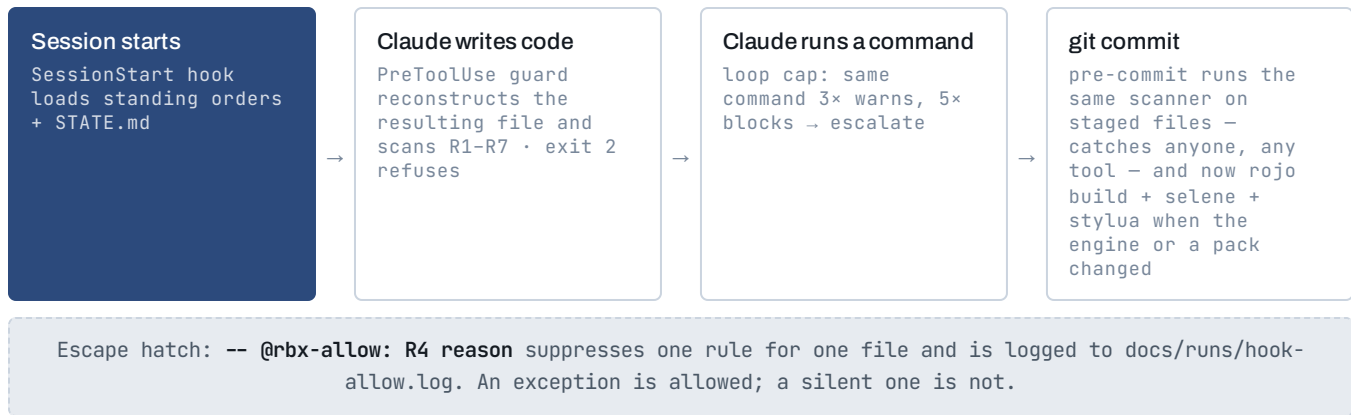


Figure 10 – Where enforcement fires. One scanner, three triggers. Verified: zero findings across the 1,587 existing lines of engine and pack code; each rule proven to fire on a deliberately bad file; a correct receipt handler passes; the git backstop refused a real commit. → `.claude/hooks/`, `.githooks/pre-commit`, `skills/hooks.md`

The premise is untested, and it is filed that way

The whole layer rests on the claim that prose rules drift over a long session while enforced ones do not. The first scan of Justin's real code came back clean — consistent both with "prose works fine" and with "one careful run does not test drift." So the claim is filed in the ledger as **E-0005, n=0, an open hypothesis**, where it cannot be cited as a reason for anything. If the guard never fires across three titles, the layer is ceremony and should be simplified. Part 4 returns to this.

2.6 The evidence ledger

Justin asked for evidence *"recorded in context to that situation and then compared"* — a learning matrix, not a filing cabinet. A closeout written as prose cannot be compared to anything. So every finding is a record carrying seven fixed context dimensions, because a finding without its context is a rumour: *"offline earnings beat daily login"* is not true in general. It was true for a snack-shape collect game aimed at 8–12 during a rising wave. Strip the context and it becomes confident bad advice.

ONE RECORD · EVIDENCE/LEDGER/E-####.MD

- **Claim**
one sentence, specific enough to be wrong
- **Seven context tags**
domain · loop verb · session shape · audience · monetisation · wave · scale
- **Outcome**
confirmed · refuted · inconclusive · observed
- **Metric**
what actually settled it
- **Confidence**
n=0 untested · n=1 one game · n=2 reproduced → eligible to become a default
- **Source, date, status**
active · superseded · retired (with reason) · open-hypothesis

ASKING IT · EVIDENCE/MATCH.PY

- **Give it a brief or a context**
--brief games/<slug>/brief.md
- **It returns matching records, ranked**
and WHY each one matched – so it can be overruled
- **Flags every single observation**
"n=1 – NOT a law"
- **Flags stale records**
older than 12 months → re-verify
- **Warns of unresolved contradictions**
touching any retrieved record
- **Says so when nothing matches**
"this is new ground – file what you try"

Figure 11 – The evidence ledger. Built and running. At intake, before any design work, the ledger is asked what it knows about this context, and the expansion step is seeded by history instead of a blank page. Explainable on purpose: a match you cannot interrogate is one you cannot argue with.

THREE RULES THAT STOP IT ROTTING

1. **Confidence is always visible.** One observation from one game is never a law. Nothing becomes a default until a *different* title reproduces it. This stops the ledger laundering a single accident into doctrine.
2. **Contradictions are kept, never overwritten.** When a new title refutes an old record, both stand, linked. A contradiction is the most valuable thing in the ledger: two honest results that disagree mean *a context dimension is missing* – something varied between those games that nobody was tracking. Finding that variable is how the matrix gets smarter. Overwriting the loser destroys exactly that information.
3. **Records go stale.** Roblox changed discovery, ads, gambling policy and testing tools inside fifteen months. Anything older than a year is flagged on retrieval; records killed by a platform change are retired with the reason, never deleted.

CONTRADICTIONS: FILE ALWAYS, ASK RARELY

Justin's instruction: a conflict comes to him as a question rather than being ignored or auto-resolved – "*less automation but more accuracy.*" Adopted, with one refinement he approved: escalating *every* conflict would make him the bottleneck, and most conflicts ask him something he has no basis to answer. "*Did it fail because of the audience or because the wave was fading?*" is not owner knowledge; asking produces a guess with his name on it.

A contradiction does not need resolving when it is found. It needs resolving when a decision depends on it.

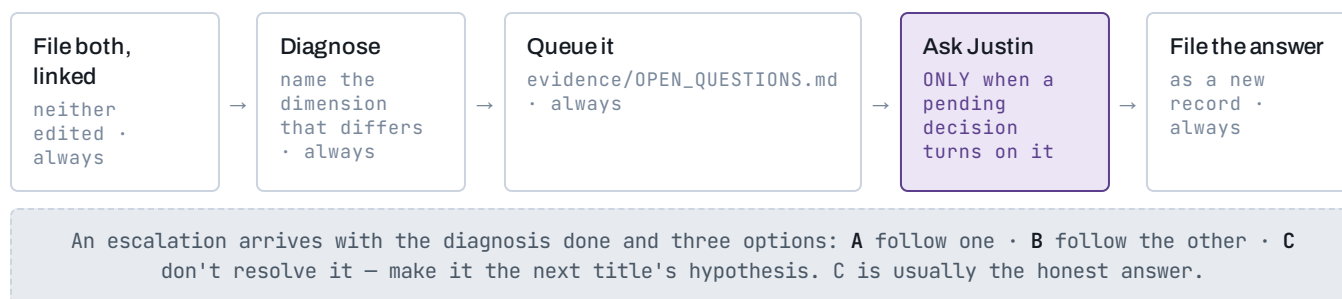


Figure 12 – The contradiction protocol. Verified against a synthetic pair: it named `wave` as the missing variable with no hints. The queue is currently empty — expected, since contradictions need two titles to have reported on the same thing. An empty queue means the ledger is young, not that the system works.

2.7 The compounding loop — how the factory learns

Justin's most important instruction of the day: *"part of your program is learning. Once you learn you record — and new sessions will learn from it and make changes and new tests to improve the process."* There is a hard constraint underneath it that shapes the whole design:

Claude does not persist between sessions

A session ends and everything it worked out is gone. So "Claude learns from it" can only ever mean one thing mechanically: **the session writes what it learned into the folder, and the next session is *forced* to read it.** Not "should read it" — forced. That is why the SessionStart hook exists. Memory that depends on someone remembering to look is not memory.



Figure 13 – The compounding loop. Today the four returns do not happen – a title ends, a document is written, nothing reads it. The ledger gives them somewhere to land; the mechanisms below make them happen.

#	MECHANISM	WHAT IT DOES	STATUS
1	SessionStart load	Every session begins holding the standing orders, the control panel (STATE.md : the standing rule, where we are, what is waiting on Justin, the next move) and the latest market digest	BUILT
2	The ledger + matcher	Structured, context-scoped, queryable evidence with the contradiction protocol	BUILT
3	Closeout	A title cannot end without a structured record the next title reads — ≤10 fields, mostly auto-filled from gate results, not an essay	MISSING
4	Hypotheses per title	1–2 predictions in the brief, instrumented at the gates, marked confirmed / refuted / inconclusive at closeout. Without a prediction, a closeout is storytelling	MISSING
5	Scheduled research → folder	The Monday digest now runs bound to the Mac, reads STATE.md for the active title, checks the platform against the doctrine's own claims, and writes a dated drop to research/ plus one line to DIGEST_LOG.md , which SessionStart surfaces. The old chat-only task is disabled	REWIRED
6	Session-close hook	A session cannot end without recording what changed and why	MISSING

RECORDING IS NOT LEARNING – EVERY TITLE CARRIES A TEST

A closeout that only reports what happened produces stories, not knowledge. Given any outcome, a clever enough session can explain why it was always going to happen — that is how a system convinces itself it is learning while the games stay identical. So every title declares one or two hypotheses *before* it is built, specific enough to be wrong: "We think

offline earnings will beat a daily-login reward on D1 for a snack-shape game." A prediction that failed is worth more than five pages of retrospective, and refuted tests are where the next title's tests come from. This is the "new tests" half of Justin's instruction.

THE AUTONOMY CONTRACT

"Make changes on your own" needs a boundary or it collides with the gates. The line is **reversibility**: anything that can be read, argued with and undone is autonomous. Anything that spends money, ships to players, or changes what the factory is aiming at is not.

CLAUDE CHANGES ON ITS OWN

CLAUDE PROPOSES, JUSTIN DECIDES

Add a hook rule after an incident a rule missed	Remove or weaken an existing rule
Raise a gate bar or move a budget cap that evidence moved	Lower a gate bar
Move twice-written code into the shared engine	Kill or switch the active title
Write research drops, pattern teardowns, evidence records	Promote an idea to the active title
Write the closeout and update STATE.md	Publish, or create any paid product
Correct a factual claim in a doc that went stale	Change a core doctrine position
Log an idea to IDEA_LOG.md	Anything that spends money

Every autonomous change is appended to `evidence/CHANGELOG.md` with the evidence that caused it — append-only, never rewritten. That is how a system that modifies itself stays auditable by someone who was not in the room. The failure it prevents is quiet drift: four titles from now, nobody — including Claude — knowing what the rules are supposed to be or which were ever tested.

2.8 Portability and the boundary

Justin will run multiple games, probably under different Claude projects, and needs the engine, rules, process, evidence and craft to be *already there* when a new title starts. Today the factory and the first title share a folder, so nothing travels. Two things follow, one done and one deliberately deferred.

Done: the boundary is enforced. What makes a factory portable is not where the folders sit; it is that the engine knows nothing about any particular game. A generic engine in an ugly directory is portable. A game-aware engine in a beautiful directory is not. Hook R7 refuses any engine module that names a game, so the engine stays generic by force rather than by intention.

Deferred: the physical move. Lifting the factory to the folder root rewrites every path and the Rojo configuration — breaking the only working repo in service of a second title that does not exist. The doctrine says do not build for a game you do not have. The move happens when a second title actually needs it and can say what it needs to support. It will cost more then; that cost is real and smaller than breaking today's repo for a speculative benefit.

Non-negotiable when it does happen: evidence has one home. If title 2 writes its learnings into its own project folder, title 3 never sees them and every project quietly relearns the same lessons in parallel. All closeouts, research and changelog entries write back to the single `evidence/` store at the root, whichever project produced them. That folder already lives at the root for exactly this reason.

FLEXIBLE WITHOUT BEING FORMLESS

Two of Justin's requirements pull against each other and the tension is real: *"the same questions and the same filters every time"* versus *"each game will have different issues — Claude can't be stuck on an old process."* The resolution is that not everything in the process is the same kind of thing.

	INVARIANTS	DEFAULTS
What	Never change, whatever the game	The starting position, changed freely per game
Examples	Server-authoritative economy · no third-party IP · no simulated gambling · publishing and paid products are a human click · a complexity budget exists · an onboarding artifact exists · a closeout gets written	<i>Which</i> six questions · <i>what</i> the caps are · which systems · which return hook · which research applies
Changing one	Requires Justin, explicitly	A title just does it, and says so in one line

The deviation rule is what makes the process learn instead of ossify: a title that departs from a default declares it in one line with a reason, exactly like the existing spec-deviation rule. Those declarations land in the evidence store. **When the same deviation shows up in two titles, it stops being a deviation and becomes the new default.** A rigid-looking system is never more than two games away from correcting itself. A process that cannot be deviated from gets abandoned the first time it does not fit; a process that can be deviated from silently is not a process.

2.9 Who does each job best

Justin's standing rule, made concrete: the best tool wins, not Claude by default.

JOB	BEST TOOL	CLAUDE'S ROLE
Market stats, rankings, revenue estimates	RoMonitor Stats · RoWatcher · Rolimon's · RTrack (free)	Read and synthesise
Luau, architecture, economy maths, docs	Claude	Own it
Quick in-Studio generation	Roblox Assistant + Cube 3D (free, first-party)	Direct it via MCP
Thumbnails and icons — the #1 click lever	Ideogram / Recraft / Grok Imagine, or a Talent Hub artist when it matters	Spec and judge; cannot render
3D props	Studio gen first → Meshy / Tripo / Hunyuan3D	Spec, import, wire

JOB	BEST TOOL	CLAUDE'S ROLE
Music	Creator Store licensed library (free, zero licensing risk)	Pick and insert
A/B testing	Roblox native Experiments	Design and read them
Kid and creator discourse	TikTok, YouTube comments, DevForum; X via Grok if already paid for	Mine and synthesise
Is it fun? Is it understood?	Real kids, Hunter operating — irreplaceable	Protocol and observation
Scheduled research and ops	Claude scheduled tasks (inside the existing plan)	Running — see Part 4 for the fix it needs

PART 3 · AGENTS, MECHANISMS AND TASKS

Who and what does the work, and how they connect

Justin asked for the list. Here it is in full — every human, AI, automated mechanism and Roblox-side tool that exists today, plus the ones that are designed but not yet built — followed by the map of how they connect and the one rule that keeps them from colliding.

3.1 The roster

AGENT / MECHANISM	KIND	WHAT IT DOES	STATUS
Justin	Human	Owner. The five gate decisions. The cut. Everything that spends money or ships. Resolves escalated contradictions. ~12 minutes a weekday.	HUMAN
Hunter	Human	Design input. Operator of the comprehension test — recruits fresh testers, runs the verbatim script, records answers. Test subject only for titles he did not help design.	HUMAN
Fresh testers (3+ kids)	Human	The only valid instrument for "is it understood." Never seen the game, no design input. Rotates per title.	HUMAN
Claude — primary engineer	AI coworker	The only writer to <code>src/</code> and the live Studio DataModel. Research, design, economy, Luau, docs, orchestration. Proposes every applicable idea. Files evidence. Works within the autonomy contract.	ACTIVE
Grok chat	AI coworker	Research, art sheets, clip hooks, second pass on design and economy, critique, live-ops copy. X/Twitter meme velocity if Justin already has access. New standing job: the confusion pass before every cut — a different model has different blind spots. Never writes to the repo or Studio.	ACTIVE
Grok Build · Grok Bots · Automations	AI (evaluated)	Second coding agent and paid automation tiers. Not adopted: cost and coordination overhead without a capability Claude lacks. Revisit at 3+ live games.	NOT ADOPTED
SessionStart hook	Mechanism	Loads the standing orders, the live hook list, <code>STATE.md</code> , idea-log and exception counts into every Claude Code session before it does anything.	BUILT
PreToolUse guard (R1–R7)	Mechanism	Scans every file Claude is about to write, reconstructing the result so whole-file rules work on a one-line edit. Refuses on R1–R4, R7; warns on R5–R6.	BUILT

AGENT / MECHANISM	KIND	WHAT IT DOES	STATUS
Debug-loop cap	Mechanism	Same shell command 3× in a session warns; 5× blocks and demands a written escalation to Justin instead of a sixth attempt.	BUILT
Git pre-commit backstop	Mechanism	Runs the same scanner on staged files at every commit — catches code written by any agent, any tool, or by hand.	BUILT
Evidence matcher (match.py)	Mechanism	Ranked, explained retrieval from the ledger; contradiction detection with the missing dimension named; matrix and open-questions generation.	BUILT
Weekly Roblox market digest	Scheduled task	Mondays 8am, bound to the Mac: movers, clone waves and their stage, platform changes, one opportunity, and a doctrine check — anything found that contradicts a doctrine claim, quoted with its source. Writes to <code>research/</code> ; never touches anything else.	REWired
Studio MCP (built into Studio)	Roblox-side tool	Claude's hands in Studio: read/edit scripts, run Luau, start/stop play, screenshots, console, simulated input, mesh/material/texture generation, Creator Store insert. Inspect and playtest — never the primary home of code.	WIRED
Playtest subagent (Roblox)	Roblox-side tool	Runs gameplay scenarios inside Studio and verifies outcomes. The candidate for automated Gate A smoke tests (join → first earn → first spend → first upgrade).	AVAILABLE · NOT WIRED
Roblox Assistant · Cube 3D · Experiments · Creator Hub analytics	Roblox-side tool	First-party generation, native A/B testing, and the analytics ground truth. Used through MCP or in-Studio; Claude designs and reads, never re-implements.	AVAILABLE
Open Cloud	Roblox-side tool	Publishing and product catalogue. Human-gated, always. No Claude action reaches it.	G5 ONLY
studio_sync.py	Mechanism (workaround)	Serves the disk tree as JSON so Studio can pull it without a human click on the Rojo plugin — the only unattended disk → Studio path found in run 01. A test harness, not a Rojo replacement.	WORKAROUND
Intake agent	Planned	Four paragraphs → the six-question brief, drafted for Justin to correct rather than fill.	MISSING
Expansion agent	Planned	Reads the ledger and pattern library, proposes every applicable mechanic. Claude and Grok both contribute. Nothing rejected at this step.	MISSING
Budget + cut-list generator	Planned	Applies the caps, ranks what is left, draws the recommended line, writes the parking list.	MISSING
Onboarding agent	Planned (engine)	Builds the walkthrough / intro / how-to every game must ship with. A missing engine module, not a per-game task.	MISSING

AGENT / MECHANISM	KIND	WHAT IT DOES	STATUS
Closeout writer + session-close hook	Planned	A title cannot end without a structured closeout; a session cannot end without logging what changed. Both feed the ledger and changelog.	MISSING

3.2 How they connect

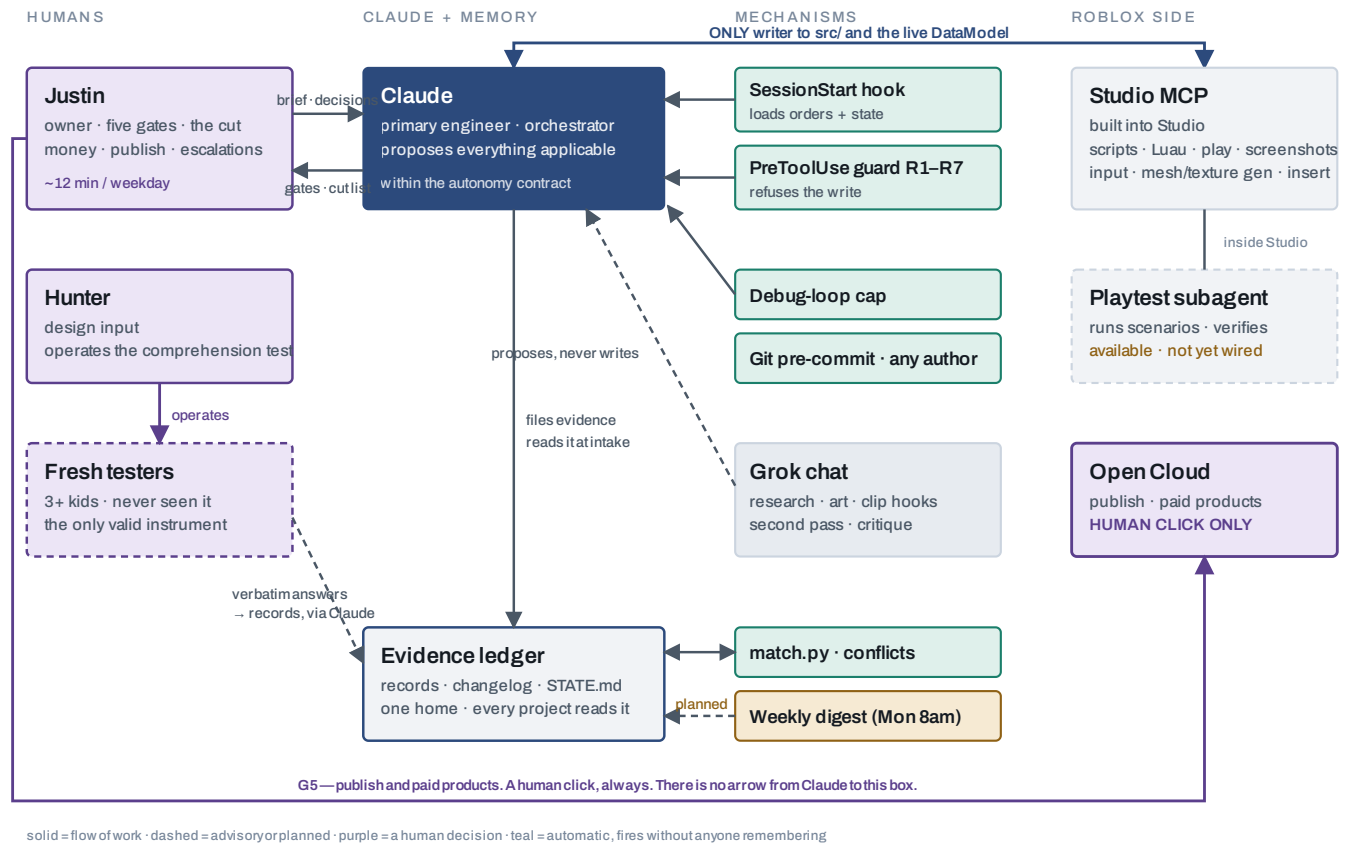


Figure 14 – The connection map. Read the arrows into Claude: humans give it briefs and decisions, hooks load it and refuse it, Grok advises it, the ledger informs it. Read the arrows out: it writes to Studio and to the ledger, and hands gates and cut lists back to Justin. The one arrow it does not have is to Open Cloud.

3.3 The one-writer rule

Everything in the map obeys one constraint that is not negotiable for speed: **Claude is the only writer to `src/` and to the live Studio DataModel**. Two agents editing Studio at once corrupts the DataModel. Grok researches and draws while Claude writes — in parallel, on disk and out-of-band, never in Studio. This is why Grok Build was not adopted and why every hand-off in the pipeline is a document, not a shared editor.

3.4 The tasks, phase by phase, and who owns each

PHASE	TASK	OWNER	JUSTIN'S PART
Kickoff	Six questions, once: age band, scope, monetisation stance, art seed, budget caps, deadline hook — or four paragraphs the intake agent turns into them	Justin → Claude	~60 seconds
R · Research	Market map from the trackers, loop autopsy of the top three, wave timing, policy flags, why-they-pay synthesis, three scored one-page pitches	Claude ll Grok	G1: pick one
D · Design	One-page loop diagram, first-session script, economy workbook, retention systems from the fair-play catalogue, live-ops calendar to week 8, KPI contract, 1–2 hypotheses	Claude, Grok second pass	G2: approve numbers
Expand + cut	Ledger consulted; every applicable idea proposed; budget applied; ranked cut list with a line drawn	Claude + Grok → Justin	Move the line, ~2 min
B · Build	Content pack on the engine; hooks enforce R1–R7; specs before code; the onboarding artifact	Claude alone	None
T · Test	Economy Monte-Carlo, automated playtest via MCP, unit tests, comprehension test with fresh kids	Claude · Hunter operates the test	G3 fun-check, 10 min
A · Art	Icon, thumbnail, one screenshot; readable at 200px; no third-party IP	Claude specs, external tools render, Grok drafts	G4: approve
Gate B	Unlisted soft launch to 200–500 strangers; retention table vs the contract; iterate the hook up to three times	Claude measures, proposes	Read the table. Kill or proceed is a gate decision
G5	Publish. Create paid products.	Justin	The click
L · Launch	Shorts and TikTok clips, Moments on the experience page, micro-creators in the genre, then small Gate C ads on the winning icon	Claude drafts, Justin approves spend	One-line approvals
O · Live-ops	Monday digest → Friday KPI review → next-drop spec → build → publish. Weekly micro-drop, monthly theme, quarterly tentpole. Monthly economy audit	Claude runs the loop	One-line yes each week, G5 click
Closeout	Hypotheses marked, evidence filed, deviations recorded, engine candidates extracted, budget questions answered	Claude	None — but no new title starts without it

What Justin's week actually looks like when this runs

Monday: read the market digest, two minutes. **Any day:** a gate decision arrives as a one-page pitch, a spreadsheet, or a ranked list with the line already drawn — reply in one line, or say nothing and defaults proceed (except G5, which waits forever). **When a slice is ready:** ten minutes playing it with Hunter while Hunter separately runs three fresh kids through the script. **Occasionally:** a contradiction that a real decision depends on, arriving with the diagnosis done and three options. **Never:** babysitting a RemoteEvent, choosing a price ladder from scratch, or being asked to invent a mechanic. The doctrine's twelve-minute review window is the design constraint, and every escalation is shaped to fit inside it.

PART 4 · THE RE-EXAMINATION

Does it make sense? Where could it hurt a good game?

Justin asked for the whole system to be worked through again. This part does that with no salesmanship: what holds, where each mechanism could detract from a good game and what stops it, the places our own documents contradict each other, and the risks from the first real build that still outrank anything added today.

4.1 What holds

- **The doctrine is sound and was already ahead of anything published.** A survey of public Claude-for-Roblox scaffolding found knowledge libraries — how to write a DataStore, genre templates — and nothing that decides when to stop, what proves a game is worth money, or what kills a title. The doctrine had all three before today.
- **Every mechanism added today is title-agnostic.** None of the seven rules, the ledger schema, the budget, or the comprehension test mentions a game. A new title inherits all of it without anyone remembering to apply it.
- **The division of labour is coherent and matches the owner.** AI proposes, Justin cuts. The owner is asked only for the kind of judgment he actually has and never for the kind he said he does not.
- **Autonomy has a boundary that is the right one.** Reversibility. Claude changes what can be undone; Justin decides what cannot.
- **The stack is cheap.** \$0 additional spend; the expensive alternatives were evaluated and declined for stated reasons.

4.2 Where each mechanism could detract, limit, or distract — and what stops it

MECHANISM	HOW IT COULD HURT A GOOD GAME	WHAT STOPS IT	RESIDUAL RISK
Safety hooks R1–R7	A false block interrupts a build; a rule fires on legitimate code	Zero false positives across all 1,587 existing lines; each rule tested both ways; escape hatch with an audit log; new rules must pass the two-way test before landing	Low. The premise itself is untested (E-0005)
Complexity budget	Caps a great idea that needed five systems; makes a first slice too thin to be fun	Constrains the <i>first slice</i> only; nothing is deleted — parking list; a declared deviation is one line and two deviations make a new default; caps expire at the first Gate A and move on evidence	Medium until the first Gate A moves them. This is the mechanism to watch

MECHANISM	HOW IT COULD HURT A GOOD GAME	WHAT STOPS IT	RESIDUAL RISK
The gates	Slow shipping; Gate B needs 200–500 strangers, which is real effort for a two-person studio	By design — the doctrine's whole thesis is that spending before Gate B rents empty visits. Unlisted soft launch, DevForum and Discord tester pools, and Hunter's cohort widen the pool for free	Real. Gate B is the most labour-intensive step and has never been run
Comprehension test	Recruiting three fresh kids per Gate A is a logistical burden; a bad day skews the result	Hunter operates and his cohort rotates — a kid is fresh for each new title; 2-of-3 thresholds absorb one outlier	Medium. Practical, not conceptual
Evidence ledger	False confidence from one observation; or nobody files anything and it dies quietly	Confidence always visible; nothing becomes a default below $n=2$; no veto power; the closeout requirement and session-close hook (not yet built) are what keep it fed	High until the feeders exist. A ledger nobody writes to is a graveyard with a schema
Closeout + changelog	Paperwork per title; the ritual becomes the work	The closeout is a template of about ten fields, most auto-filled from gate results and hypotheses — never an essay. The changelog is append-only one-liners	Low, if the template stays short
Contradiction protocol	Justin becomes the bottleneck; or conflicts pile up unread	Escalate only when a pending decision depends on one; otherwise they wait in a visible queue; every escalation arrives with the diagnosis done and "make it the next test" as an option	Low
Fixed interview and filters	Rigidity — a game that does not fit the six questions gets forced into them	Invariants versus defaults: the questions are defaults and can be deviated from in one declared line	Low

The largest risk to this project tonight is not on that table

It is that **the system becomes the project**. Today was one hundred percent system and zero percent game. That was the right call for one day — the scaffolding needed to exist — and it is exactly the pattern the doctrine warns about if it continues: *"feature-complete before anyone has played,"* applied to process instead of content. A factory that produces documents about itself is the anti-pattern with better typography.

The rule this document proposes, and the one it most wants Justin to hold it to: no further system work — no new hooks, no new agents, no restructuring — until *Fat Man Gets Rich* has been through Gate A with the comprehension test and three fresh kids. The only exceptions are the items in Part 5 that directly unblock that test. The test of the factory is a game. Everything built today is a hypothesis until one has gone through it.

4.3 Contradictions found between our own documents

Working through the folder again surfaced six places where the documents disagree with each other or with what Justin said today. None is fatal. All of them would confuse a future session that read the wrong one first.

#	THE CONFLICT	RESOLUTION	WHO ACTS
1	Three different D1 scales. The doctrine (Gate B) says ~12%+ is OK to test small spend and under 8–10% is death. PIPELINE §4's KPI contract says $D1 \geq 20\%$ to proceed, $\geq 25\%$ to spend on ads. SYSTEM-MAP §9 says kill under 15% after three tries, scale at 25%+. A game at 13% is simultaneously "OK to spend a little," "do not proceed," and "iterate or kill."	The doctrine governs by its own precedence rule. Its numbers are the <i>floor</i> . Done: PIPELINE §4 and SYSTEM-MAP §9 now say the 20/25% figures are tiers, not gates.	FIXED · Justin confirms the floor stands
2	"The human owns taste" (doctrine §0 and §9) versus Justin today: <i>"I'm not sure I can supply the taste."</i>	Read the doctrine's "taste" as <i>the cut</i> — subtractive judgment, which Justin does supply. Propose a one-line amendment to §9 so a future session does not ask him to invent mechanics.	Claude proposes; Justin decides (a doctrine position)
3	Doctrine §3 lists "extra agents" as effort that does not count. This document proposes five new agents.	The doctrine's target is production capacity — more workers making more content, the "40th ore type" class. The agents proposed here remove work from the owner (intake, cut list, closeout) or are required by a quality bar (onboarding). The distinction is real but the tension is honest: each new agent must justify itself against §3 before it is built, and the rule above (no system work until Gate A) applies to all of them.	Standing test, applied by Claude, visible to Justin
4	PIPELINE §7's "Hunter protocol" has Hunter playing the game across three sessions. The comprehension skill says Hunter cannot be the primary subject for a title he helped design.	Two different tests, both valid. The Hunter protocol is the G3 <i>fun-check</i> ; the comprehension test is Gate A's <i>is-it-understood</i> check and needs fresh subjects. Done: the skill file now says so.	FIXED
5	The weekly market digest exists but is disconnected. It runs Mondays in a fresh session with no access to the folder, its prompt says "no game is in active development," and its output lands only in a chat window — which the folder's own first rule says does not exist as a deliverable.	Done: re-created bound to the Mac; reads <code>STATE.md</code> ; writes a dated drop to <code>research/</code> and a line to <code>DIGEST_LOG.md</code> ; runs a doctrine check each week. The old task is disabled, not deleted. SessionStart surfaces the latest line.	FIXED · first run Monday 8am

#	THE CONFLICT	RESOLUTION	WHO ACTS
6	Small inconsistencies from run 01, still open: two naming conventions (GAME_FACTORY lowercase pack files vs rojo-map PascalCase); engine UI code sitting in the game's client folder with no engine mount; the Phase 0 checklist in PIPELINE §13 still shows "decide first target" unchecked although Fat Man Gets Rich was chosen.	Done: naming ruled in rojo-map; checklist ticked; studio_sync and the MCP-require rule written into the studio-mcp skill. Still open: the <code>src/core/client/</code> mount, before the second game.	MOSTLY FIXED

4.4 The risks from the first build that outrank today's work

Run 01's own "blunt list of what was missing" is the most useful document in the folder for this question, and most of it is still open. These threaten the *game* directly, which none of today's system work does, and several of them should be fixed before the next hook is written.

#	RISK	WHY IT OUTRANKS SYSTEM WORK	FIX · OWNER
1	Persistence is unverified. Studio cannot reach DataStores until the place is published, so the save system fell back to memory. Kill-the-server-and-rejoin has never been run.	Data loss is the #1 Gate A killer in the doctrine. Every hook in R4 protects a path that has never executed.	Publish an <i>unlisted</i> place so the DataStore path can be exercised. This is Gate-5-adjacent and is Justin's click. Then Claude runs acceptance test 8
2	Economy simulated — and it found the problem. <code>tools/econ_sim.py</code> runs 300 players against the real config and the engine's own formulas. Result: first upgrade at 28 seconds (right), but the entire 27,925-coin ladder is maxed by day 3, offline earnings are 82% of all coins, and a kid holds 1.38 million unspent by day 30. The second week is empty and playing is pointless versus waiting.	This is exactly the "week-2 billionaires" failure, caught as a spreadsheet problem. It is the single most important game-facing finding of the day.	FOUND · Candidate C proposed for G2 (offline 0.5/hr, deeper ladders): nothing maxed inside 30 days, offline ~47%, first minute untouched. Justin approves the numbers. Late-ladder wall remains — needs a second sink, a live-ops decision
3	No test framework, no CI enforcement. TestEZ is named in the plan and not installed; nothing makes <code>rojo build</code> , Selene and StyLua run on a commit.	The first edit to a cost curve breaks something silently.	HALF DONE · pre-commit now runs rojo build + selene + stylua on the Mac. TestEZ still not installed; the sim is the only economy check
4	No unattended path from disk into Studio. The Rojo plugin needs a human click in the editor that MCP cannot make. <code>studio_sync.py</code> is a workaround, not documented in the skill.	Every automated playtest depends on it. Run 01 called it the single highest-leverage fix in the repo.	DOCUMENTED in <code>skills/studio-mcp.md</code> , with the two other run-01 rules. A cleaner path through the built-in MCP is still worth a look

#	RISK	WHY IT OUTRANKS SYSTEM WORK	FIX · OWNER
5	No art seed, no sound. The brief says TBD; the flood's readability at 200px rests on "neon yellow on a dark floor."	Blocks G4. Not urgent before Gate A, and the doctrine says grey-box first — but the thumbnail is the #1 click lever and has no owner yet.	Justin: one reference image or three adjectives, when Gate A passes

4.5 Untested premises, stated plainly

- **That prose rules drift and hooks do not** (E-0005, n=0). The enforcement layer rests on it. If no guard fires across three titles, simplify the layer.
- **That the complexity caps are near the right values.** They are guesses. The first comprehension test is the first evidence.
- **That three fresh ten-year-olds can be recruited per Gate A.** Hunter's cohort makes it plausible; it has not been done.
- **That evidence with seven context tags is comparable across titles.** The dimensions were chosen deliberately, and the first real contradiction will tell us whether one is missing.
- **That a two-hour "create a ___ game" is real.** GAME_FACTORY says it becomes real on the *next* game, now that the engine exists. It has never been timed.
- **The ledger holds zero player evidence.** Every record is about how the factory behaves. Nothing is known about real players because no title has met one. That is an accurate report, not a gap in the ledger — but it means every design conversation so far has been theory.

PART 5 · STATUS AND BUILD ORDER

Where it stands, and what happens next

5.1 Status board



Figure 15 — Status board, 12 September 2026, after the second pass. Teal built, amber partial, red missing. Seven of fifteen built, four partial, four missing — ordered below.

5.2 The build order

Ordered by one principle: **get a real game through a real Gate A first, and build only what unblocks that until it has happened.** Everything after step 3 waits for the comprehension test's results, because those results are the first evidence the factory will ever hold about players, and several later items should be shaped by them.

#	MOVE	OWNER	SIZE	WHY IN THIS POSITION
1	Publish an unlisted place; verify persistence	Justin clicks, Claude tests	Small	Unblocks the #1 Gate A killer. Nothing else about the slice can be trusted until saves survive a server restart

#	MOVE	OWNER	SIZE	WHY IN THIS POSITION
2	Economy Monte-Carlo; build + lint in the pre-commit; TestEZ	Claude	DONE except TestEZ	The sim found the day-3 exhaustion; Candidate C is on Justin's list for G2. Build + lint now run on every Mac commit
3	Gate A — the comprehension test on Fat Man Gets Rich	Hunter operates · 3 fresh kids · Justin fun-checks	One afternoon	The first real evidence. Answers the budget's two questions. Proves or refutes the system's premise in one sitting
4	Closeout template, hypotheses in the brief template, session-close hook, changelog wiring	Claude	Small	Gate A produces evidence with nowhere structured to land. Build the feeders right after the first thing worth feeding
5	Re-create the weekly digest bound to the folder	Claude	DONE	First folder-bound run Monday 8am. The old chat-only task is disabled
6	The intake loop — paragraphs → brief, expansion inventory, budget as code, ranked cut list	Claude	Medium	Gap 0. The stated purpose of the project. Positioned after Gate A so the first cut list is shaped by what real kids actually found confusing
7	Onboarding agent as an engine module	Claude	Medium	Deliberately after Gate A, not before. Fat Man Gets Rich was designed to teach itself — the first payout overfills the bag, which is supposed to teach banking with no tutorial. Testing the raw slice is the purest test of that design. If fresh kids understand it with no walkthrough, the onboarding agent's job is small; if they don't, we learn exactly what needs explaining. Building the agent first would hide the answer
8	Fix the six document contradictions (§4.3)	Claude; Justin on #2	DONE except #2	#2 is a doctrine wording — the exact line is in STATE.md for a yes / no / edit
9	Visual quality bar; craft library home	Claude proposes, Justin's art seed	Medium	Blocks G4, not Gate A. "Visually professional" needs a definition before art enters the repo at volume
10	Title registry; physical factory split	Claude	Medium	When the second title exists and can say what it needs. Not before

5.3 Decisions that are genuinely Justin's

Only the irreversible ones. Everything else in this document runs on stated defaults. The same list lives in `STATE.md`, which every session loads, so it cannot be forgotten.

1. **Publish an unlisted place** so persistence can be tested. It is not a launch — unlisted, link-only — but it is a publish action, and the doctrine makes every publish a human

click. docs/runs/PERSISTENCE_TEST.md has the three clicks (including the one everyone forgets: enabling Studio API access) and the seven-row test Claude runs the moment it is done.

2. **G2 — the economy numbers.** The simulation proposes Candidate C. Approve, adjust, or reject before the kids see it; the slice can be tested un-tuned, but the numbers in front of them should be the ones we mean.
3. **Two doctrine wordings:** the proposed \$9 line is in STATE.md verbatim — *"Human owns: ship / no-ship, money, and the cut — removing what confuses. Generative taste is not asked of the owner; the AI proposes, the owner prunes."* Yes, no, or edit. And confirm the doctrine's Gate B floor governs; both other files now say it does.
4. **Hold the line in §4.2:** no further system work until Fat Man Gets Rich has been through Gate A with fresh testers. This is the decision that protects the project from its own scaffolding, and it is the one Claude cannot make for you.

The answer to the question this document was written for

What has been created will improve games and learn from your work *if it is now used to make one*. It does not limit a good idea: nothing is deleted, every cap expires, every default can be deviated from in a line, and the ledger has no veto. It does not distract from an opportunity: ideas are pitched the moment they appear and logged without pausing the active title, and killing a title is a gate decision with a closeout, never a mid-build hop. Where it could detract — false blocks, a cap that is too tight, a ledger nobody feeds, or a system that keeps building itself — the safeguard is named, and where the safeguard does not exist yet, it is in the build order with an owner.

The factory is built enough. The next thing it should produce is evidence, and evidence only comes from a game in front of a stranger.

APPENDIX A

Where everything lives

The `Roblox Business` folder on the Mac is the source of truth. The `claude.ai` project "RB – Business" is a read-only mirror. One canonical path per file; everything else links to it.

PATH	WHAT IT IS
START-HERE.md	The map of the folder and the routing rule for where new files go
system-map.html · roblox-pipeline/SYSTEM-MAP.md	The pipeline drawn as ten diagrams (render · source)
evidence/	The factory's memory. SCHEMA.md, ledger/E-####.md, match.py, MATRIX.md, OPEN_QUESTIONS.md, CHANGELOG.md
.claude/hooks/ · .claude/settings.json	rbx_guard.py (R1–R7), pretooluse_guard.py, loop_cap.py, session_start.py; hook registration
.githooks/pre-commit	The git backstop (core.hooksPath is set)
GrokBDownloads/	Inbound only — approved Grok art (fbx, glb, previews)
roblox-pipeline/CLAUDE.md	What Claude loads at the start of every build session
roblox-pipeline/PIPELINE.md	The operating plan: what happens when you say "create a ___ game"
roblox-pipeline/docs/ROBLOX_SUCCESS_LOGIC.md	Standing orders. Outranks every other file
roblox-pipeline/docs/SYSTEM_ARCHITECTURE.md	The factory across many titles: layers, build loop, compounding, portability, autonomy
roblox-pipeline/docs/GAME_FACTORY.md	Engine / content-pack split; the six-question brief; the two-hour clock
roblox-pipeline/docs/BUDGET.md	The complexity caps — provisional defaults with an expiry
roblox-pipeline/docs/FACTORY_PLAN.md	This document, canonical text
roblox-pipeline/docs/IDEA_LOG.md	Every game idea. Logged → Shortlisted → Active → Shipped Parked
roblox-pipeline/docs/runs/	One log per build session; STATE.md (read into every session); hook-allow.log
roblox-pipeline/skills/	Ten binding conventions: security, economy, art, live-ops, research-legal, rojo-map, studio-mcp, idea-intake, hooks, playtest-comprehension
roblox-pipeline/research/	Dated genre drops, the research playbook, patterns/ (mechanic teardowns)
roblox-pipeline/specs/	Module specs, written before code, from TEMPLATE.md
roblox-pipeline/src/core/	The engine. Eleven services. A new game must not change it (R7 enforces)
roblox-pipeline/games/<slug>/	Content packs. One folder per title — brief, config, world, sku, theme. The only per-game work
roblox-pipeline/tools/	build_map.mjs (renders the system map), studio_sync.py (unattended disk → Studio), meshimport/

Appendix B — The comprehension script, verbatim

1. **Say nothing.** Hand over the device. Do not explain, do not point.
2. **Watch for 60 seconds.** Record: did they take a meaningful action unprompted? Which one? How long?
3. **Ask, in these words:** *"What are you trying to do in this game?"* Write the answer verbatim.
4. **Ask:** *"Do you want to play it again?"* Never "did you like it" — liking is politeness.
5. **Only then** may the operator answer questions or explain anything.

Appendix C — One evidence record, as written

```

---
id: E-0005
claim: Prose non-negotiables drift over long build sessions in a way that machine-enforced
ones do not
domain: process          loop_verb: any          session_shape: any          audience: any
monetization: any        wave: any              scale: pre-launch
outcome: inconclusive
metric: UNTESTED. The guard found zero true violations across 1,587 existing lines
confidence: n=0
observed: 2026-09-12
status: open-hypothesis
---
This record exists to be proven wrong. Filed at n=0 deliberately so it cannot be cited as a
reason for anything.
```

The record the enforcement layer's own premise is filed under. A ledger that only records its wins is not evidence; it is advertising.

Appendix D — Glossary for a non-programmer

Agent / subagent	An AI instance given one narrow job. A subagent is spun off by another agent, does its task, reports back, and disappears.
CCU	Concurrent users — how many people are in a game at the same moment. The number trackers show; not the number discovery rewards.
CI	Continuous integration — checks that run automatically on every change (build, lint, tests) so nothing broken lands quietly.
Closeout	The short structured record a title produces when it ships or dies: hypotheses marked, evidence filed, deviations noted.
Clone wave	A genre that many studios copy at once after one hit. Rises in weeks, fades in months. Enter early or not at all.
Content pack	Everything specific to one game — names, numbers, map, products, art — in one folder. The engine reads it at boot.

D1 / D7	The share of new players who come back the next day / a week later. The number Roblox's discovery system now weights most.
DataStore	Roblox's save system. The place where "a kid loses everything" happens if it is done wrong.
DevEx	Developer Exchange — converting earned Robux to money. About \$0.0038 per Robux.
Engine	The reusable code every game shares — saving, security, economy, progression. Written once, never per game.
Gate	A checkpoint where the work stops for a decision or a measurement. Five are human decisions (G1–G5); Gates A/B/C are evidence bars.
Hook	A small program that runs automatically on an event — before a file is written, when a session starts — and can refuse the action. A rule with a trip-wire instead of a sentence.
Ledger	The evidence store: one record per finding, with the context that says when it applies.
Luau	The programming language Roblox games are written in.
MCP	Model Context Protocol — the plug that lets Claude reach into a running program. With Studio's MCP, Claude can read scripts, run code, press play and take screenshots instead of only writing text for a human to paste.
Onboarding / FTUE	The first-time experience — the walkthrough, intro or how-to that gets a new player to understanding.
Pre-commit	A check that runs when code is saved into the project's history. Ours refuses the save if a rule is broken.
Remote / RemoteEvent	How a player's device talks to the server. The place cheaters attack. Every one is validated and rate-limited.
Robux	Roblox's currency. Kids buy it with money; games earn it; DevEx converts it back.
Rojo	The tool that keeps the game's code on disk (where it can be reviewed and versioned) and syncs it into Studio.
Session (Claude)	One conversation. Claude keeps nothing between sessions except what is written to a file that the next session is made to read.
Skill	A file of binding conventions Claude loads when a task matches. A reference shelf, not a trip-wire.
SKU / product	One thing for sale in the game — a pass, a pack, a cosmetic.
Slice	The smallest playable version of a game: one loop, one shop, one return hook. What Gate A tests.
Studio	Roblox Studio — the editor where games are built and playtested.

Appendix E — Sources compiled

ROBLOX_SUCCESS_LOGIC.md (doctrine) · PIPELINE.md · SYSTEM-MAP.md · GAME_FACTORY.md · SYSTEM_ARCHITECTURE.md · BUDGET.md · skills/hooks.md · skills/playtest-comprehension.md · docs/runs/2026-09-12-run-01.md · docs/runs/STATE.md · IDEA_LOG.md · evidence/SCHEMA.md and ledger E-0001–E-0005 · evidence/CHANGELOG.md · research/2026-09-12-claude-scaffolding-survey.md · the scheduled-task registry · the session of 12 September 2026 with Justin.

The Game Factory · v1.0 · 12 September 2026 · Re-verify platform facts by 12 November 2026.